

PRACTICAL 1



CHARUSAT[®]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY

AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL 1

Practical 1: Array Operations: Traversal, Rotation, and Frequency Analysis

Problem 1

A bakery prepares n items every morning and places them in a display row. At the end of each hour, the leftmost item is moved to the rightmost position to make room for fresh stock at the front. Given the initial row and the number of hours h , print the final display order.

Describe the approach you used. How does it behave when h is very large, say 10 million? Is there a way to get the correct result without performing the operation h times?

Theory:

1. Why does the display order repeat?

The display order repeats because there are only a fixed number of n items. Every hour, the items are shifted left by one position. After n hours, every item returns to its original position. Therefore, the arrangement repeats every n hours.

2. What determines when it first repeats?

The number of items n determines the repetition period. After exactly n rotations, the original arrangement is obtained again. Therefore, instead of performing h rotations, we can calculate $h \% n$ and perform only the remaining rotations.

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical%201

Output screenshot:

```
1 2 3 4 5
Enter the number of items: 12

The arrangement of the bread in the shelves will be:
3 4 5 1 2
```

Description:

PRACTICAL 1

In this practical , we keep item in a shelf and every hour the back most item comes to front . So in x hours of time which item will be at front and what will be its arrangement.

Challenges/Difficulties:

I faced difficulty in implement the logic of the code how to give rotation to the items.

Github Repository link:

https://github.com/kanan2007s-sys/FDSA_B

Conclusion:

The program successfully rotates the bakery items by moving the leftmost item to the rightmost position. The arrangement repeats after n rotations, so using $h \% n$ can avoid performing unnecessary rotations when h is very large.

PRACTICAL 1

Problem 2

A library issues books to students and records each book's ID every time it is borrowed. At the end of the month, the librarian wants to find all books that were borrowed more than once, as those need priority restocking. Given the borrowing log, print all such book IDs.

Describe the approach you used. How many times does your solution scan the data? If the library had 100,000 borrow records, would your approach still be practical? Can you think of a way that requires fewer passes?

Theory:

1. If the same book ID appears three times, should it appear once or three times?

It should appear once in the output because the purpose is to identify which book IDs were borrowed more than once. The number of repetitions is used for counting, not for printing the ID multiple times.

The given program maintains a separate count for each book ID and increments it whenever that ID appears.

2. Does the solution handle this correctly? How can you verify it?

Yes, the counting approach handles repeated IDs by increasing their count. We can verify it by entering the same book ID three times and checking that its count becomes 3. The ID itself should still be considered only once when identifying repeated books.

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical%201

Output Screenshot:

PRACTICAL 1

```
Enter 10 Book IDs: 101
101
102

102
102
103
102
104
104
105
Most taken book ID: 102
```

Description:

In this practical we need to find the most books taken and count of the book issued from library.

Challenges/Difficulties:

I faced difficulty in using the logic , that how to store number of counts of every books borrowed from library and implementation of it.

Github Repository link:

https://github.com/kanan2007s-sys/FDSA_B

Conclusion:

The program counts how many times each book ID appears in the borrowing log. Repeated book IDs can be identified using their counts. Although the nested-loop approach works for a small number of records, a frequency table or hash map can reduce the number of comparisons and make the solution more efficient for large data.

PRACTICAL 1

Problem 3

A school newspaper editor wants to highlight the most impressive word from a submitted article

on the front page. The word is chosen simply by length — the longest one wins. Given a sentence, print the winning word and how many letters it has.

Describe the approach you used. What does your solution do when two words have the same length? Is that behavior intentional, and can you think of a way to handle it explicitly?

Theory:

The editor later decides that punctuation attached to a word (e.g. “hello,” or “great!”) should not count toward its length. How would this change your solution, and at which step would you handle it?

The program finds the longest word in a sentence by reading each word one by one and comparing its length with the length of the current longest word. If the current word is longer, it becomes the new longest word.

If two words have the same length, the program keeps the first word encountered because it only updates the longest word when a word has a strictly greater length. This behavior is intentional in the current implementation. If required, the program could be modified to handle ties explicitly, for example by printing all words having the maximum length.

If punctuation such as ,, ., or ! should not count toward the word length, it should be removed before calculating and comparing the word length.

Code:

https://github.com/kanan2007s-sys/FDSA_B/tree/main/Practical%201

Output screenshot:

```
Enter a sentence: This sentence is very long in length
The longest word is: sentence
```

PRACTICAL 1

Description:

In this practical we need to find the longest word from the headline of the newspaper .

Challenges/difficulties faces:

In this practical I couldn't know that which header file should be used and how to use the stringstream function and its syntax. Then I sorted out that it will give words one by one.

Repository link:

https://github.com/kanan2007s-sys/FDSA_B

Conclusion:

In these three problems, basic array and string processing techniques are used to solve practical data-handling tasks. The first problem demonstrates that repeated rotations are cyclic, so large values of h can be reduced using $h \% n$. The second problem demonstrates frequency counting of book IDs and shows that a single-pass approach using a frequency table or hash map is more efficient for large data. The third problem demonstrates string processing to find the longest word, with punctuation removable before calculating word length. Overall, these problems illustrate how choosing an appropriate algorithm can improve both correctness and efficiency, especially when the input size becomes large.